

Singleton Design Pattern Revision Notes

Definition

Singleton is a creational design pattern that restricts a class to exactly one instance and provides a controlled access point to that instance.

Simple meaning: only one object should exist for a particular responsibility inside the process.

Supporting context

Static members

A static field or method belongs to the class, not to one object.

Singleton usually uses:

- A static field to store the one instance.
- A static method such as `getInstance()` to return that instance.

Private constructor

In Java, a private constructor prevents external code from calling `new`.

```
private AppLogger() {  
}
```

This means only the class itself can create its instance.

Interface overview

An interface is a contract that describes behavior without forcing callers to know the concrete class.

Example:

```
interface Logger {  
    void info(String message);  
}
```

Business services can depend on `Logger`, while the actual implementation can be `AppLogger`.

This is useful because Singleton can make code tightly coupled. Interfaces and dependency injection reduce that coupling.

Lazy initialization

Lazy initialization creates the singleton only when it is first requested.

Benefit: avoids startup cost if the object is never used.

Risk: in multi-threaded languages, lazy creation must be thread safe.

Eager initialization

Eager initialization creates the singleton when the class loads.

Benefit: simple and thread-safe in Java.

Risk: creates the object even if it is never used.

JavaScript module caching

In JavaScript, ES modules are evaluated once and cached. Exporting one object from a module is often the cleanest singleton style.

When to use

Use Singleton when:

- Exactly one instance should exist in the process.
- Object creation is expensive and repeated creation is wasteful.
- A shared coordinator must keep consistent state.
- The object has no user-specific or request-specific state.
- Lifecycle should be controlled from one place.

Avoid Singleton when:

- The object stores request, user, tenant, or transaction data.
- Tests need independent instances.
- The object has mutable business state.
- A normal dependency-injected service would be clearer.

Where to use

Good candidates:

- Application configuration
- Logger
- Metrics registry
- Feature flag provider
- Telemetry collector
- Connection pool manager
- Cache manager

Poor candidates:

- Shopping cart
- User session
- Order entity
- Payment transaction
- Request context
- Tenant-specific mutable state

JavaScript example: Classic Singleton

```
class AppLogger {
  static instance;

  static getInstance() {
    if (!AppLogger.instance) {
      AppLogger.instance = new AppLogger();
    }
  }
}
```

```

        return AppLogger.instance;
    }

    constructor() {
        if (AppLogger.instance) {
            return AppLogger.instance;
        }

        this.logs = [];
    }

    info(message) {
        this.logs.push({ level: "info", message });
        console.log(message);
    }
}

const logger = AppLogger.getInstance();
logger.info("Order confirmed");

```

JavaScript example: Module Singleton

```

// logger.js
class Logger {
    info(message) {
        console.log("[info]", message);
    }
}

export const logger = new Logger();

// orderService.js
import { logger } from "./logger.js";

export function confirmOrder(order) {
    logger.info("Order confirmed: " + order.id);
}

```

This is usually more idiomatic in JavaScript than forcing a Java-style singleton class.

Java example: Basic Singleton

```

interface Logger {
    void info(String message);
}

public final class AppLogger implements Logger {
    private static AppLogger instance;

    private AppLogger() {
    }

    public static AppLogger getInstance() {
        if (instance == null) {
            instance = new AppLogger();
        }

        return instance;
    }

    public void info(String message) {
        System.out.println(message);
    }
}

```

Note: this lazy version is not thread safe.

Java example: Thread-safe holder Singleton

```

public final class MetricsRegistry {
    private MetricsRegistry() {
    }
}

```

```

private static class Holder {
    private static final MetricsRegistry INSTANCE =
        new MetricsRegistry();
}

public static MetricsRegistry getInstance() {
    return Holder.INSTANCE;
}

public void record(String name, double value) {
    // record metric
}
}

```

The holder idiom is lazy and thread safe because Java class loading is thread safe.

Pros

- Ensures one controlled instance.
- Can reduce repeated expensive initialization.
- Provides consistent shared configuration.
- Useful for infrastructure services such as logging and metrics.
- Gives a clear lifecycle point for process-wide resources.

Cons

- Can become hidden global mutable state.
- Can make unit tests harder.
- Can hide dependencies from constructors.
- Can violate Single Responsibility if it mixes global access and business behavior.
- Thread safety can be tricky in multi-threaded environments.
- Lifecycle issues can appear in long-running applications.

Business logic efficiency

Singleton can help business logic when it keeps infrastructure consistent and cheap.

Example: application configuration.

Without Singleton:

- Many services load config separately.
- Values may be inconsistent if reload timing differs.
- Startup and file parsing cost repeats.

With Singleton:

- Config loads once.
- Services read the same values.
- Business services avoid configuration construction details.

Business benefits:

- Consistent operational behavior.
- Centralized logging and metrics.

- Less duplicated initialization.
- Cleaner service classes.
- Fewer integration mistakes.

Important caution: Singleton should usually support business logic, not contain core mutable business data.

Interview question and answer

Q1. What is the Singleton design pattern?

Answer: Singleton is a creational pattern that restricts a class to one instance and provides a global access point to that instance.

Q2. Why is Singleton controversial?

Answer: It can become hidden global state. This makes dependencies less visible, tests harder, and behavior more coupled.

Q3. Lazy vs eager Singleton?

Answer: Lazy Singleton creates the instance on first use. Eager Singleton creates it when the class loads. Lazy can save startup cost, while eager is simpler and often thread safe.

Q4. How do you make Singleton thread safe in Java?

Answer: Use enum Singleton, eager initialization, synchronized access, double-checked locking with volatile, or the initialization-on-demand holder idiom.

Q5. How is Singleton commonly implemented in JavaScript?

Answer: Often by exporting a single object from a module, because modules are evaluated once and cached.

Q6. How can interfaces help with Singleton?

Answer: Services can depend on an interface such as `Logger` instead of directly depending on `AppLogger.getInstance()`. This improves testing and replacement.

Scenario-based question and answer

Scenario 1

Question: A config file is loaded separately by many services and values become inconsistent. Is Singleton useful?

Answer: Yes. A read-only `ConfigManager` singleton can load configuration once and provide consistent process-wide access.

Scenario 2

Question: A shopping cart is stored as a Singleton because the app should have one cart. Is this correct?

Answer: No. A shopping cart is user-specific state. Singleton would mix users or requests. Use session-scoped or request-scoped objects.

Scenario 3

Question: A Java server uses a lazy Singleton for metrics. Many threads may call `getInstance()` together. What is the risk?

Answer: Without thread safety, multiple instances may be created. Use holder idiom, enum singleton, or another thread-safe approach.

Scenario 4

Question: A logger singleton causes tests to fail because logs leak between tests. What should you do?

Answer: Prefer injecting a Logger interface. Keep the singleton stateless if possible. If needed, expose a controlled reset only for test infrastructure.

Problem-solving question and answer

Problem 1

Question: Implement one shared config reader in JavaScript.

Answer:

1. Create a ConfigManager class.
2. Instantiate it once in config.js.
3. Export that instance.
4. Import it wherever configuration is needed.

```
// config.js
class ConfigManager {
  constructor() {
    this.values = Object.freeze({
      paymentProvider: process.env.PAYMENT_PROVIDER
    });
  }

  get(key) {
    return this.values[key];
  }
}

export const config = new ConfigManager();
```

Problem 2

Question: Make a Java lazy Singleton thread safe without synchronizing every call.

Answer: Use the initialization-on-demand holder idiom.

```
public final class AppConfig {
  private AppConfig() {
  }

  private static class Holder {
    private static final AppConfig INSTANCE = new AppConfig();
  }

  public static AppConfig getInstance() {
    return Holder.INSTANCE;
  }
}
```

Problem 3

Question: Existing Singleton hides dependencies and blocks unit testing. How do you refactor?

Answer:

1. Create an interface for the behavior.
2. Make the singleton class implement the interface.
3. Inject the interface into business services.
4. Use the singleton only at the application composition root.

5. In tests, inject a fake implementation.

Problem 4

Question: Singleton stores mutable tenant data and production has cross-tenant bugs. What should you do?

Answer: Remove Singleton for tenant data. Use tenant-scoped, request-scoped, or explicit context objects. Singleton should not store data that varies per user, request, tenant, or transaction.

Fast memory hooks

- Singleton means one instance.
- It is a creational pattern.
- Use it for process-wide infrastructure.
- Avoid it for mutable business data.
- Java uses private constructor plus static access.
- JavaScript often uses module exports.
- Thread safety matters in multi-threaded languages.
- Dependency injection is often cleaner than direct global access.